

Lista de Exercícios 11 - Lista Simplesmente Encadeada

1. Crie as seguintes funções:

Observação: É proibido o uso de estrutura de dados auxiliares e arrays.

```
//insere "valor" após a primeira ocorrência de um valor par na lista
void insereAposPrimeiroPar(No **lista, int valor);

//insere "valor" antes da primeira ocorrência de um valor par na lista
void insereAntesPrimeiroPar(No **lista, int valor);

//retorna o total de elementos da lista
int totalL(No **lista)

//verifica se duas listas são iguais
bool igualL(No **lista1, No **lista2)

//Insere um valor no final da lista
void insereFinalL(No **lista, int valor)

//retorna o valor do último elemento da lista SEM removê-lo
int leFinalL(No **lista)

//retorna o valor do último elemento da lista e REMOVE o elemento
int removeFinalL(No **lista)

//retorna o valor do dado armazenado em um nó da lista, com base na posição passada por parâmetro
//se posição=0, a função deve retornar o valor do primeiro nó da lista
//se posição=1, a função deve retornar o valor do segundo nó da lista
int lePosicao(No **lista, int posição)

//insere "valor" antes de CADA ocorrência de um valor par na lista
void insereAposCadaPar(No **lista, int valor);
```

2. Crie uma lista encadeada para gerenciar Produtos, sendo que cada produto é composto por nome e quantidade. Crie as seguintes funções:
- `soma(lista, nome)`: deve retornar a soma da quantidade de produtos (somar o campo quantidade de cada produto) cuja o nome foi passado por parâmetro.
 - `contProdutos(lista, nome)` : deve retornar a quantidade de elementos da lista possuem o nome passado por parâmetro.
 - `somaIntervalo(lista, I, F)`: retorna soma das quantidades dos produtos cujo o intervalo é definido pelos parâmetros I e F, que que são os elementos da lista que estão entre as posições I e F,
 - `combinarProd(lista, nome)`: combinar os produtos que possuem o mesmo nome, somando a quantidade dos mesmos. Exemplos:

```
{[melancia, 3], [melao, 2], [melao, 10], [melancia, 5], [abacaxi, 6]}
```



```
Fila: {tamanho = 30, total = 3}  
      {[melancia, 8], [melao, 12], [abacaxi, 6]}
```
 - `verificaIntervalos(lista1, lista2, I1, F1, I2, F2)`: retorna verdadeiro caso a soma das quantidades dos produtos de ambos os intervalos seja igual; caso contrário, deve retornar falso. Os intervalos de cada lista são definidos pelos parâmetros.
3. Um grupo de pesquisa em segurança da informação está estudando a frequência de uso de senhas semelhantes pelos usuários de um determinado sistema. Assim, foi criada uma lista contendo as senhas dos usuários. Uma das análises a ser realizada é a verificação de senhas não seguras. Uma senha pode ser considerada “não segura” se ela possuir uma quantidade muito pequena de caracteres ou for utilizada por uma quantidade muito grande de usuários. Para classificar as senhas não seguras, você deve criar uma lista simplesmente encadeada contendo todas as senhas com menos de 4 caracteres ou cuja frequência de utilização é maior que 5 (ou seja, senhas que aparecem mais que cinco vezes na lista). Crie a função `naoSeguras` que recebe uma lista de senhas e retorna outra lista contendo todas as senhas não seguras existentes.

4. Faça uma função que verifica se uma lista encadeada está ordenada:

```
int verificaOrdem(No** lista);
```

Retorno da função:

- 0 – Ordem crescente
- 1 – Ordem decrescente;
- 2 – Não está ordenada.

Observação: É proibido o uso de listas auxiliares, percorrer os nós da lista.

5. Construa uma função para ordenar uma lista simplesmente encadeada.

```
void ordenarL(No** lista);
```

Observação:

- É proibido o uso de listas auxiliares, percorrer os nós da lista.
- A lista não pode estar vazia.

Dicas:

- Percorrer os nós da lista, verificando o dado do nó atual e do próximo (caso ele exista), trocando os valores dos dados dos nós caso necessário.
- Uma lista está completamente ordenada quando não há mais trocas de dados entre os nós, ou seja, ao percorrer uma lista, se nenhuma troca foi realizada, a lista está ordenada.

6. Crie uma função para inserir em uma lista encadeada com base em uma posição/índice.

Observação: É proibido o uso de listas auxiliares.

```
bool inserePosicao(No** lista, int pos, int valor);  
L1: 2 5 3 9 9 8 7 6 2 1
```

```
inserePosicao(L1, 0, 100); //returna true  
L1: 100 2 5 3 9 9 8 7 6 2 1
```

```
inserePosicao(L1, 3, 200); //returna true  
L1: 100 2 5 200 3 9 9 8 7 6 2 1
```

```
inserePosicao(L1, 11, 300); //returna true  
L1: 100 2 5 200 3 9 9 8 7 6 2 300 1
```

```
inserePosicao(L1, 13, 1000); //returna false, pois pos é inválido  
L1: 100 2 5 200 3 9 9 8 7 6 2 300 1
```

```
inserePosicao(L1, -1, 1000); //returna false, pois pos é inválido  
L1: 100 2 5 200 3 9 9 8 7 6 2 300 1
```